

宁波大学
NINGBO UNIVERSITY

宁波大学信息学院

实验报告

实验课程名称： 嵌入式系统

学　　院： 信息科学与工程学院

专　　业： 计算机科学与技术

指 导 教 师： 王翀,王艺睿

报告人姓名： 顾凯琪 **学号：** 226001624

班　　级： 2022 级计算机 2 班

学　　期： 2024-2025 学年第二学期

信息学院实验中心制

实验								
成绩								

实验项目名称	**实验十一 嵌入式 Linux 多线程编程实验**

一、实验目的与要求：

1.1 实验目的

(1) 理解 Linux 系统中线程的基本概念，包括线程与进程的区别和联系。

(2) 学习使用 POSIX 线程库（pthread）进行多线程编程的基本方法。

(3) 掌握线程的创建、同步、通信以及取消等操作。

1.2 实验要求

(1) 熟悉 pthread 库提供的线程创建、管理和同步的 API 函数，包括 pthread_create、pthread_join、pthread_mutex_*等。

(2) 编写程序实现多线程的创建，理解线程属性设置和线程的生命周期管理。

(3) 学习线程间的同步机制，包括互斥锁（mutex）、条件变量（cond）和信号量（semaphore）的使用。

(4) 掌握线程间的数据共享和通信方法，编写程序实现线程间的数据交换和状态同步。

(5) 学习线程的取消和清理，编写程序演示如何安全地取消一个线程并进行资源回收。

二、实验设备：

(1) 华清远见开发环境。

(2) 实验平台为华清远见提供的虚拟机环境。

(3) 使用的操作系统是 Ubuntu 12.04 LTS 64-bit。

(4) 虚拟机工具软件为 VMware Player。

三、实验方法（原理、流程图）

3.1 实验原理：

3.1.1 线程基本编程

本实验的线程操作均在用户空间中进行。Linux 系统中常用的 pthread 线程库是一套符合 POSIX 标准的通用线程库，具有良好的可移植性。

a) 线程的创建和启动：

使用 pthread_create()函数创建线程，该函数指定线程函数的入口点。创建线程后，线程开始执行指定的线程函数。线程函数执行完毕后，线程自动退出。

b) 线程的退出:

线程可以通过 pthread_exit()函数主动退出。

在线程中不应使用 exit()函数进行出错处理，因为 exit()会导致整个进程终止，包括该进程中的所有线程。相反，应使用 pthread_exit()函数安全退出线程。

c) 资源回收:

由于进程中的多个线程共享数据段，线程退出后，其占用的资源不会自动释放。

使用 pthread_join()函数可以等待线程结束并回收其资源。该函数会阻塞调用它的线程，直到被等待的线程结束。

d) 线程的取消:

pthread_cancel()函数用于请求取消另一个线程。

被取消的线程需要使用 pthread_setcancelstate()和 pthread_setcanceltype()函数设置其取消状态和取消类型，以决定是否响应取消请求，以及如何响应。

关键函数概述

1. pthread_create(): 创建线程。

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
void *(*start_routine)(void *), void *arg);
```

头文件：#include <pthread.h>
参数：
thread：线程标识符。
attr：线程属性设置，通常为 NULL。
start_routine：线程函数的起始地址。
arg：传递给线程函数的参数。
返回值：成功返回 0，出错返回错误码。
2. pthread_exit(): 主动退出线程。

```
void pthread_exit(void *retval);
```

头文件：#include <pthread.h>
参数：
retval：线程结束时的返回值，可由 pthread_join()获取。

3. pthread_join(): 等待线程结束并回收资源。

```
int pthread_join(pthread_t th, void **thread_return);
```

头文件：#include <pthread.h>
参数：
th：等待线程的标识符。
thread_return：指向存储被等待线程结束时返回值的指针。
返回值：成功返回 0，出错返回错误码。

4. pthread_cancel(): 请求取消线程。

```
int pthread_cancel(pthread_t th);
```

头文件：#include <pthread.h>
参数：
th：要取消的线程的标识符。
返回值：成功返回 0，出错返回错误码。
通过这些函数，可以实现对线程的创建、管理和取消操作，确保线程在多线程环境下能够安全、有效地运行并正确处理资源的分配和回收。

3.1.2 线程之间的同步与互斥

由于线程共享进程的资源和地址空间，因此在对这些资源进行操作时，必须考虑到线程间资源访问的同步与互斥问题。这里主要介绍 POSIX 中两种线程同步机制，分别为互斥锁和信号量。这两个同步机制可以互相通过调用对方来实现，但互斥锁更适合用于同时可用的资源是唯一的情况；信号量更适合用于同时可用的资源为多个的情况。

a) 互斥锁线程控制

互斥锁是用一种简单的加锁方法来控制对共享资源的原子操作。这个互斥锁只有两种状态，也就是上锁和解锁，可以把互斥锁看作某种意义上的全局变量。在同一时刻只能有一个线程掌握某个互斥锁，拥有上锁状态的线程能够对共享资源进行操作。若其他线程希望上锁一个已经被上锁的互斥锁，则该线程就会挂起，直到上锁的线程释放掉互斥锁为止。可以说，这把互斥锁保证让每个线程对共享资源按顺序进行原子操作。

互斥锁机制主要包括下面的基本函数。

- 互斥锁初始化：pthread_mutex_init()

- 互斥锁上锁：pthread_mutex_lock()
- 互斥锁判断上锁：pthread_mutex_trylock()
- 互斥锁接锁：pthread_mutex_unlock()
- 消除互斥锁：pthread_mutex_destroy()

其中，互斥锁可以分为快速互斥锁、递归互斥锁和检错互斥锁。这三种锁的区别主要在于其他未占有互斥锁的线程在希望得到互斥锁时是否需要阻塞等待。快速锁是指调用线程会阻塞直至拥有互斥锁的线程解锁为止。递归互斥锁能够成功地返回，并且增加调用线程在互斥上加锁的次数，而检错互斥锁则为快速互斥锁的非阻塞版本，它会立即返回并返回一个错误信息。默认属性为快速互斥锁。

（1）线程基本编程

这里要讲的线程相关操作都是用户空间中的线程的操作。在 Linux 中，一般 pthread 线程库是一套通用的线程库，是由 POSIX 提出的，因此具有很好的可移植性。

创建线程实际上就是确定调用该线程函数的入口点，这里通常使用的函数是 pthread_create()。在线程创建之后，就开始运行相关的线程函数，在该函数运行完之后，该线程也就退出了，这也是线程退出的一种方法。另一种退出线程的方法是使用函数 pthread_exit()，这是线程的主动行为。这里要注意的是，在使用线程函数时，不能随意使用 exit()退出函数进行出错处理，由于 exit()的作用是使调用进程终止，往往一个进程包含多个线程，因此，在使用 exit()之后，该进程中的所有线程都终止了。因此，在线程中就可以使用 pthread_exit()来代替进程中的 exit()。

由于一个进程中的多个线程是共享数据段的，因此通常在线程退出之后，退出线程所占用的资源并不会随着线程的终止而得到释放。正如进程之间可以用 wait()系统调用来同步终止并释放资源一样，线程之间也有类似机制，那就是 pthread_join()函数。pthread_join()可以用于将当前线程挂起来等待线程的结束。这个函数是一个线程阻塞的函数，调用它的函数将一直等待到被等待的线程结束为止，当函数返回时，被等待线程的资源就被收回。

前面已提到线程调用 pthread_exit()函数主动终止自身线程。但是在很多线程应用中，经常会遇到在别的线程中要终止另一个线程的的问题。此时调用 pthread_cancel()函数实现这种功能，但在被取消的线程的内部需要调用 pthread_setcancel()函数和 pthread_setcanceltype()函数设置自己的取消状态，例如被取消的线程接收到另一个线程的取消请求之后，是接受还是忽略这个请求；如果是接受，则再判断立刻采取终止操作还是等待某个函数的调用等。

表 1.1 列出了 pthread_create()函数的语法要点。

表 1.1　pthread_create()函数语法要点

所需头文件	#include <pthread.h>
函数原型	int pthread_create ((pthread_t *thread, pthread_attr_t *attr, void *(*start_routine)(void *), void *arg))
函数传入值	thread：线程标识符
	attr：线程属性设置（其具体设置参见 6.4.3 小节），通常取为 NULL
	start_routine：线程函数的起始地址，是一个以指向 void 的指针作为参数和返回值的函数指针
	arg：传递给 start_routine 的参数
函数返回值	成功：0
	出错：返回错误码

表 1.2 列出了 pthread_exit()函数的语法要点。

表 1.2　pthread_exit()函数语法要点

所需头文件	#include <pthread.h>
函数原型	void pthread_exit(void *retval)
函数传入值	retval：线程结束时的返回值，可由其他函数如 pthread_join()来获取

表 1.3 列出了 pthread_join()函数的语法要点。

表 1.3　pthread_join()函数语法要点

所需头文件	#include <pthread.h>
函数原型	int pthread_join ((pthread_t th, void **thread_return))
函数传入值	th：等待线程的标识符
	thread_return：用户定义的指针，用来存储被等待线程结束时的返回值（不为 NULL 时）
函数返回值	成功：0
	出错：返回错误码

表 1.4 列出了 pthread_cancel()函数的语法要点。

表 1.4　pthread_cancel()函数语法要点

所需头文件	#include <pthread.h>
函数原型	int pthread_cancel((pthread_t th)
函数传入值	th：要取消的线程的标识符
函数返回值	成功：0
	出错：返回错误码

（2）线程之间的同步与互斥

由于线程共享进程的资源和地址空间，因此在对这些资源进行操作时，必须考虑到线程间资源访问的同步与互斥问题。这里主要介绍 POSIX 中两种线程同步机制，分别为互斥锁和信号量。这两个同步机制可以互相通过调用对方来实现，但互斥锁更适合用于同时可用的资源是唯一的情况；信号量更适合用于同时可用的资源为多个的情况。

（3）互斥锁线程控制

互斥锁是用一种简单的加锁方法来控制对共享资源的原子操作。这个互斥锁只有两种状态，也就是上锁和解锁，可以把互斥锁看作某种意义上的全局变量。在同一时刻只能有一个线程掌握某个互斥锁，拥有上锁状态的线程能够对共享资源进行操作。若其他线程希望上锁一个已经被上锁的互斥锁，则该线程就会挂起，直到上锁的线程释放掉互斥锁为止。可以说，这把互斥锁保证让每个线程对共享资源按顺序进行原子操作。

互斥锁机制主要包括下面的基本函数。

- 互斥锁初始化：pthread_mutex_init()
- 互斥锁上锁：pthread_mutex_lock()
- 互斥锁判断上锁：pthread_mutex_trylock()
- 互斥锁接锁：pthread_mutex_unlock()
- 消除互斥锁：pthread_mutex_destroy()

其中，互斥锁可以分为快速互斥锁、递归互斥锁和检错互斥锁。这三种锁的区别主要在于其他未占有互斥锁的线程在希望得到互斥锁时是否需要阻塞等待。快速锁是指调用线程会阻塞直至拥有互斥锁的线程解锁为止。递归互斥锁能够成功地返回，并且增加调用线程在互斥上加锁的次数，而检错互斥锁则为快速互斥锁的非阻塞版本，它会立即返回并返回一个错误信息。默认属性为快速互斥锁。

表 1.5 列出了 pthread_mutex_init()函数的语法要点。

表 1.5　pthread_mutex_init()函数语法要点

所需头文件	#include <pthread.h>	
函数原型	int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *mutexattr)	
函数传入值	mutex：互斥锁	
函数传入值	Mutexattr	PTHREAD_MUTEX_INITIALIZER：创建快速互斥锁
		PTHREAD_RECURSIVE_MUTEX_INITIALIZER_NP：创建递归互斥锁
		PTHREAD_ERRORCHECK_MUTEX_INITIALIZER_NP：创建检错互斥锁
函数返回值	成功：0	
	出错：返回错误码	

表 1.6 列出了 pthread_mutex_lock()等函数的语法要点。

表 1.6　pthread_mutex_lock()等函数语法要点

所需头文件	#include <pthread.h>
函数原型	int pthread_mutex_lock(pthread_mutex_t *mutex,) int pthread_mutex_trylock(pthread_mutex_t *mutex,) int pthread_mutex_unlock(pthread_mutex_t *mutex,) int pthread_mutex_destroy(pthread_mutex_t *mutex,)

函数传入值	mutex：互斥锁
函数返回值	成功：0
	出错：−1

（4）信号量线程控制

在前面已经讲到，信号量也就是操作系统中所用到的 PV 原子操作，它广泛用于进程或线程间的同步与互斥。信号量本质上是一个非负的整数计数器，它被用来控制对公共资源的访问。这里先来简单复习一下 PV 原子操作的工作原理。

PV 原子操作是对整数计数器信号量 sem 的操作。一次 P 操作使 sem 减一，而一次 V 操作使 sem 加一。进程（或线程）根据信号量的值来判断是否对公共资源具有访问权限。当信号量 sem 的值大于等于零时，该进程（或线程）具有公共资源的访问权限；相反，当信号量 sem 的值小于零时，该进程（或线程）就将阻塞直到信号量 sem 的值大于等于 0 为止。

PV 原子操作主要用于进程或线程间的同步和互斥这两种典型情况。若用于互斥，几个进程（或线程）往往只设置一个信号量 sem，它们的操作流程如图 1.1 所示。

当信号量用于同步操作时，往往会设置多个信号量，并安排不同的初始值来实现它们之间的顺序执行，它们的操作流程如图 1.2 所示。

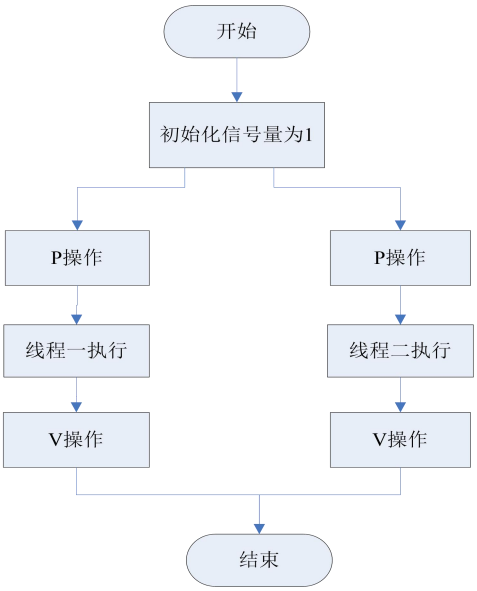


图 1.1　信号量互斥操作

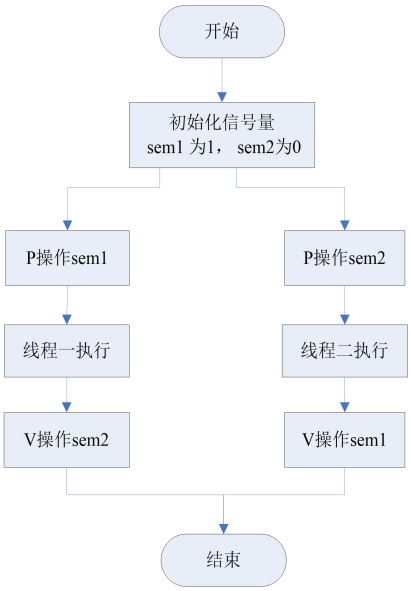


图 1.2　信号量同步操作

Linux 实现了 POSIX 的无名信号量，主要用于线程间的互斥与同步。这里主要介绍几个常见函数。

- sem_init()用于创建一个信号量，并初始化它的值。
- sem_wait()和 sem_trywait()都相当于 P 操作，在信号量大于零时它们都能将信号量的值减一，两者的区别在于若信号量小于零时，sem_wait()将会阻塞进程，而 sem_trywait()则会立即返回。
- sem_post()相当于 V 操作，它将信号量的值加一同时发出信号来唤醒等待的进程。
- sem_getvalue()用于得到信号量的值。
- sem_destroy()用于删除信号量。

表 1.7 列出了 sem_init()函数的语法要点。

表 1.7　sem_init()函数语法要点

所需头文件	#include <semaphore.h>

函数原型	int sem_init(sem_t *sem,int pshared,unsigned int value)
函数传入值	sem：信号量指针
	pshared：决定信号量能否在几个进程间共享。由于目前 Linux 还没有实现进程间共享信号量，所以这个值只能够取 0，就表示这个信号量是当前进程的局部信号量
	value：信号量初始化值
函数返回值	成功：0
	出错：−1

表 1.8 列出了 sem_wait()等函数的语法要点。

表 1.8　sem_wait()等函数语法要点

所需头文件	#include <pthread.h>
函数原型	int sem_wait(sem_t *sem) int sem_trywait(sem_t *sem) int sem_post(sem_t *sem) int sem_getvalue(sem_t *sem) int sem_destroy(sem_t *sem)
函数传入值	sem：信号量指针
函数返回值	成功：0
	出错：−1

（5）线程属性

读者是否还记得 pthread_create()函数的第二个参数（pthread_attr_t *attr）表示线程的属性。在上一个实例中，将该值设为 NULL，也就是采用默认属性，线程的多项属性都是可以更改的。这些属性主要包括绑定属性、分离属性、堆栈地址、堆栈大小以及优先级。其中系统默认的属性为非绑定、非分离、缺省 1M 的堆栈以及与父进程同样级别的优先级。下面首先对绑定属性和分离属性的基本概念进行讲解。

- 绑定属性

前面已经提到，Linux 中采用“一对一”的线程机制，也就是一个用户线程对应一个内核线程。绑定属性就是指一个用户线程固定地分配给一个内核线程，因为 CPU 时间片的调度是面向内核线程（也就是轻量级进程）的，因此具有绑定属性的线程可以保证在需要的时候总有一个内核线程与之对应。而与之对应的非绑定属性就是指用户线程和内核线程的关系不是始终固定的，而是由系统来控制分配的。

- 分离属性

分离属性是用来决定一个线程以什么样的方式来终止自己。在非分离情况下，当一个线程结束时，它所占用的系统资源并没有被释放，也就是没有真正的终止。只有当 pthread_join()函数返回时，创建的线程才能释放自己占用的系统资源。而在分离属性情况下，一个线程结束时立即释放它所占有的系统资源。这里要注意的一点是，如果设置一个线程的分离属性，而这个线程运行又非常快，那么它很可能在 pthread_create()函数返回之前就终止了，它终止以后就可能将线程号和系统资源移交给其他的线程使用，这时调用 pthread_create()的线程就得到了错误的线程号。

这些属性的设置都是通过特定的函数来完成的，通常首先调用 pthread_attr_init()函数进行初始化，之后再调用相应的属性设置函数，最后调用 pthread_attr_destroy()函数对分配的

属性结构指针进行清理和回收。设置绑定属性的函数为 pthread_attr_setscope()，设置线程分离属性的函数为 pthread_attr_setdetachstate()，设置线程优先级的相关函数为 pthread_attr_getschedparam()（获取线程优先级）和 pthread_attr_setschedparam()（设置线程优先级）。在设置完这些属性后，就可以调用 pthread_create()函数来创建线程了。

表 1.9 列出了 pthread_attr_init()函数的语法要点。

表 1.9　pthread_attr_init()函数语法要点

所需头文件	#include <pthread.h>
函数原型	int pthread_attr_init(pthread_attr_t *attr)
函数传入值	attr：线程属性结构指针
函数返回值	成功：0
	出错：返回错误码

表 1.10 列出了 pthread_attr_setscope()函数的语法要点。

表 1.10　pthread_attr_setscope()函数语法要点

所需头文件	#include <pthread.h>	
函数原型	int pthread_attr_setscope(pthread_attr_t *attr, int scope)	
函数传入值	attr：线程属性结构指针	
	scope	PTHREAD_SCOPE_SYSTEM：绑定
		PTHREAD_SCOPE_PROCESS：非绑定
函数返回值	成功：0	
	出错：−1	

表 1.11 列出了 pthread_attr_setdetachstate()函数的语法要点。

表 1.11　pthread_attr_setdetachstate()函数语法要点

所需头文件	#include <pthread.h>	
函数原型	int pthread_attr_setscope(pthread_attr_t *attr, int detachstate)	
函数传入值	attr：线程属性	
	detachstate	PTHREAD_CREATE_DETACHED：分离
		PTHREAD _CREATE_JOINABLE：非分离
函数返回值	成功：0	
	出错：返回错误码	

表 1.12 列出了 pthread_attr_getschedparam()函数的语法要点。

表 1.12　pthread_attr_getschedparam()函数语法要点

所需头文件	#include <pthread.h>
函数原型	int pthread_attr_getschedparam (pthread_attr_t *attr, struct sched_param *param)
函数传入值	attr：线程属性结构指针
	param：线程优先级
函数返回值	成功：0
	出错：返回错误码

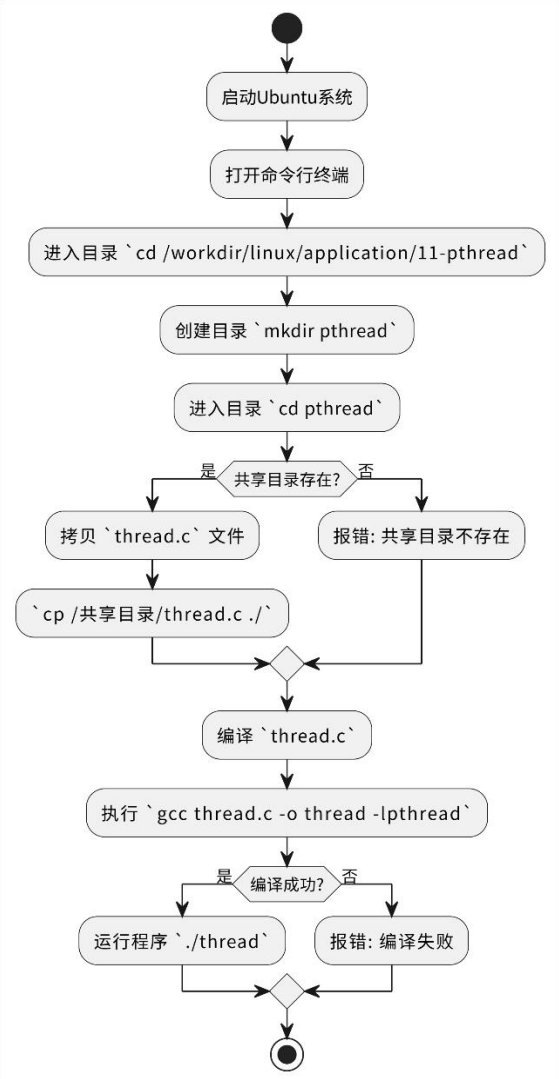
表 1.13 列出了 pthread_attr_setschedparam()函数的语法要点。

表 1.13　pthread_attr_setschedparam()函数语法要点

所需头文件	#include <pthread.h>
函数原型	int pthread_attr_setschedparam (pthread_attr_t *attr, struct sched_param

	*param)
函数传入值	attr：线程属性结构指针
	param：线程优先级
函数返回值	成功：0
	出错：返回错误码

流程图：



四、实验过程、步骤及内容

运行 Ubuntu 系统，打开命令行终端。

执行命令：

```
$ cd ~/workdir/linux/application/11-pthread
$ mkdir pthread
$ cd pthread
```

将“华清远见-CORTEXA9 资料： \实验代码\2、Linux 移植驱动及应用\1、Linux 应用\实验代码\20. Linux 系统消息队列实验\实验代码”目录下的“thread.c”拷贝到该目录下。

执行编译命令：

```
$ gcc thread.c -o thread -lpthread
```

运行程序：

$ ./thread

```
linux@ubuntu64-vm:~/workdir/linux/application/11-pthread$ cp /mnt/hgfs/share/实
验代码1/21.\ Linux系统多线程实验/实验代码/thread.c .
linux@ubuntu64-vm:~/workdir/linux/application/11-pthread$ ls
pthread  thread.c
linux@ubuntu64-vm:~/workdir/linux/application/11-pthread$ gcc thread.c -o thread
 -lpthread
linux@ubuntu64-vm:~/workdir/linux/application/11-pthread$ ./thread
dadf
```

```
linux@ubuntu64-vm:~/workdir/linux/application/11-pthread$ ./thread
dadf

welcome
***
result:welcome
```

实验代码：

thread.c

```c
#include <stdio.h>
#include <time.h>
#include <stdlib.h>
#include <errno.h>
#include <unistd.h>
#include <pthread.h>

//char buf[] = "welcome";

void *handler(void *p) //void *p = buf;
{
    getchar();
    printf("%s\n", (char *)p);

    //return p;
    pthread_exit(p);
}

void f(char **p)//char **p = &s;
{
    char *q = "dadf";
    *p = q;
}

int main(int argc, char *argv[])
{
    char  *s = NULL;
    f(&s);
    printf("%s\n", s);
```

```
30
31 #if 1
32     pthread_t thread;
33     char buf[] = "welcome";
34     void *result;
35
36     if (pthread_create(&thread, NULL, handler, (void *)buf))
37         exit(-1);
38
39     pthread_join(thread, &result);//exit(0); waitpid(pid, &status, 0);
40
41     printf("***\n");
42     printf("result:%s\n", (char *)result);
43
44     while (1);
45 #endif
46
47     exit(0);
   }
```

五、实验数据（现象）处理分析

1. 线程创建与执行

当运行程序时，程序首先输出了通过函数 f 赋值给指针 s 的字符串 "dadf"。这表明函数 f 成功地修改了指针 s 所指向的字符串内容。接下来，程序创建了一个新的线程，并传递了字符串 "welcome" 作为参数。

2. 线程等待输入

新创建的线程执行 handler 函数，该函数首先等待用户输入。此时，程序会暂停并等待我们在终端中按下任意键继续执行。这一现象显示了线程的阻塞行为，它会等待特定事件（在本例中是用户输入）发生后再继续执行。

3. 线程输出

当我们按下任意键后，线程会输出传递给它的字符串 "welcome"。这表明线程正确地接收了传递的参数，并能够在阻塞解除后继续执行。

4. 线程退出与资源回收

线程输出后，通过调用 pthread_exit 函数退出，并返回其接收到的参数值。主线程使用 pthread_join 函数等待子线程结束，并获取子线程的退出状态。在实验中，我们看到子线程的退出状态被成功地传回主线程并输出，这表明 pthread_join 成功地回收了子线程的资源。

5. 主线程持续运行

最后，程序进入无限循环，保持程序运行。这一现象展示了主线程在不显式退出的情况下，可以持续保持活跃状态，等待进一步的操作或事件。

```
linux@ubuntu64-vm:~/workdir/linux/application/11-pthread$ ./thread
dadf

welcome
***
result:welcome
```

六、实验结论：

本次实验通过对 Linux 下 pthread 线程库的学习和实践，掌握了线程的基本操作，包括线程的创建、退出、等待和取消等关键函数。通过这些操作，我们能够在多线程环境下有效地管理线程的生命周期，确保程序能够正确执行。

在多线程并发执行的过程中，线程之间的同步与互斥控制至关重要。实验采用 pthread_mutex_t 互斥锁机制来保证共享资源的安全访问，避免竞态条件的发生。通过 pthread_mutex_lock 和 pthread_mutex_unlock 控制多个线程的访问顺序，确保数据的一致性，从而提高程序的稳定性和可靠性。

此外，实验还探讨了线程的资源管理问题。通过 pthread_join 机制，我们能够正确回收线程资源，避免出现内存泄漏。同时，实验测试了 pthread_cancel 取消线程的效果，并分析了不同情况下线程的响应方式，这加深了我们对多线程安全性的理解。

总体而言，本次实验让我们熟悉了 Linux 下多线程编程的基本概念和实现方法，掌握了线程同步、互斥与资源管理等关键技术，为后续深入学习高并发编程奠定了基础。

七、指导教师批阅意见：

成绩评定：

指导教师签字：

年　　月　　日

八、备注：

注：报告内的项目或内容设置，根据实际情况可加以调整和补充，写不下可另附一页。